Software Engineering Group
Prof. Dr. Sebastian Baltes
Tobias Reichel

Heidelberg University
Summer Semester 2026

Advanced Software Engineering (IASE)

Assignment 4

Group Submissions in Pairs Allowed[1]

This assignment is about automated test-input generation ("fuzzing"). You will build a fuzzer for a small TOML parser and use it to find crashes. The parser reads a TOML file from `stdin` and exits with `0` for accepted input, `1` for rejected input, and any other exit code for a **crash**. You have access to compiled binaries, but not the source code.

The assignment repository contains a command-line fuzzing harness written in Kotlin. It provides a fuzzing loop, process execution, crash output handling, a seed corpus, a grammar, and a working `random` mode. Your task is to implement the `mutational` and `grammar` modes, find crashes, and explain the results.

Setup:

- **Clone** the assignment repository and push to a private remote of your choice, such as a private GitHub, GitLab, or bare Git repository. You can also clone and work locally, but we suggest adding a remote so that you can back up your progress.
- Use **JDK 25** and **Gradle 9.5**. The repository includes a `mise.toml`; `mise install` installs the tool dependencies. You can import the project via *File > New > Project from Existing Sources* in IntelliJ IDEA.
- The target binaries are in the `targets/toml/bin/` directory:
  - `toml_parser_linux_x86_64`
  - `toml_parser_mac_universal`
  - `toml_parser_win_x86_64.exe`

  The harness detects your operating system and uses the matching binary path listed in `targets/toml/target.yaml`.
- Run `gradle test`. Before your implementation, the test suite reports **20 failing tests** covering the functions you are asked to implement; your goal is zero failures.

Exercise 4.1 (Mutational Fuzzer: 5 points)

A *mutational* fuzzer starts from a valid seed input and edits it randomly. The `mutational` mode loads seeds from `targets/toml/seeds/`. Your task is to implement the mutators and the `mutate` function, then run the fuzzer to find crashes.

(a) **Implement mutators:** Implement `deleteRandomCharacter`, `insertRandomCharacter`, `flipRandomCharacter`, and `repeatRandomCharacter` in `mode/mutational/Mutators.kt`. Each mutator takes `input: String` and returns the edited `String`; `insertRandomCharacter` also takes `alphabet: List<Char>`. Whenever a mutator needs a random position, inserted character, bit, or repeat count, get it from `random: Random`. Do not use fixed choices, `Random.Default`, or a new `Random` instance. Two runs with the same `--random-seed` must make the same random choices in the same order. (3 points)

[1] We strongly encourage you to submit in pairs, since the tutorial presentations will also be scheduled in pairs.

(b) **Select a mutator:** Implement `MutationalFuzzer.mutate` so it applies one randomly chosen mutator. The provided `createCandidate` calls `mutate` between `minMutations` and `maxMutations` times. (1 point)

(c) **Find and report crashes:** Run `gradle run --args="--mode mutational --stop-on-crash"` repeatedly, or omit `--stop-on-crash` for longer campaigns, until you have found crashes with at least **two different exit codes**. Crashing inputs are written to `output/crashes/`, grouped into a sub-directory per exit code (e.g. `exit134/`). In `REPORT.md`, state the platform you tested on and give **one representative input per exit code**. (1 point)

Hint: Several crash exit codes are reachable with mutational fuzzing. You need to find at least two. Mutators are expected to be generic. A mutator that only replaces a hard-coded string in one seed input is not generic.

Exercise 4.2 (Grammar-Based Fuzzer: 5 points)

Mutational and purely *lexical* fuzzers are unlikely to reach bugs that require specific input structure, such as nesting deeper than the parser can handle. Nesting is *context-free*: No regular expression matches balanced brackets of arbitrary depth. A grammar-based fuzzer can generate the required structure directly.

The repository already parses `targets/toml/toml_parser.ebnf` into a small grammar AST. Grammar mode builds a derivation tree in phases. The tree starts with the grammar's start symbol. A node is *pending* while its expression has not yet been expanded; expanding it replaces that node with children derived from its expression. Helper functions handle recursive depth and termination. Implement `expandNode`, which receives one such pending node and returns its children.

(a) **Expand grammar nodes into tree children:** Implement `GrammarFuzzer.expandNode`. Use the `Expression` sealed type as the case split and return the derivation-tree children for the pending node's expression. The surrounding phase, depth, and termination logic is already provided; use the existing helper functions for strategy-dependent choices instead of duplicating that logic. (3 points)

(b) **Reach the nesting bug:** Run `gradle run --args="--mode grammar --grammar-recursive-depth 300 --grammar-recursion-strategy linear --stop-on-crash"` and confirm it crashes the binary. `--grammar-recursive-depth` sets the recursive depth target; `linear` keeps one recursive path active. (1 point)

(c) **Explain:** In `REPORT.md`, explain why a *mutational* or *lexical* (regular) fuzzer is unlikely to reach this bug and why a *grammar-based* fuzzer can. (1 point)

Optional: testing across platforms The repository includes a GitHub Actions workflow (`.github/workflows/fuzz.yml`) that runs your fuzzer against the binaries on Linux, macOS, and Windows. Using it is optional and not graded.

Submission: Use `./prepare-submission.sh` to build the ZIP file. Run it without arguments to see its usage. The script writes the archive to the parent directory. The ZIP file includes `.git/` and local crash output under `output/`; it excludes build artifacts and Gradle wrapper files. On Windows, run the script from Git Bash or WSL. The extracted ZIP file must pass `mise trust`, `mise install`, and `gradle test` from the repository root.

Submission deadline:

- Monday, 15.06.2026, 10:59

Next tutorial:

- Monday, 15.06.2026

Submission notes:

- We use the format dd.mm.yyyy for **dates** and the 24-hour format for **times**.
- Whether an **individual or group submission** is required is noted at the top of the assignment sheet. Upload your submission to **Moodle** by the deadline.
- Submission of **plagiarism** will result in a grade of **0 points** for the entire assignment. In case of **suspected** use of **GenAI tools** without attribution, the submission will also be graded with **0 points**.
- For exercises with a **written deliverable**, combine all texts, diagrams, and other parts in **a single PDF file** unless the assignment sheet specifies a different format. The first page must contain your **name** and **student ID** (Matrikelnummer); for group submissions, the names and student IDs of both members.
- For **UML exercises**, submit your **diagrams** as part of the **PDF file** (i.e., as images). You can use UMLet or PlantUML to create them. Alternatively, you can draw the diagrams by hand and scan them; or create them digitally.
- For **programming exercises**: clone the starter repository and push your work to a **private** remote of your choice (a private GitHub, GitLab, or bare Git repository). Do **not** publish your solution publicly. Submit a **zipped self-contained Git repository**; the zip must include the `.git/` directory (full commit history). Submissions without it are graded **0 points**. GitHub's "Download ZIP" strips `.git/` and is not an acceptable submission method. Exclude build artifacts (e.g., `.gradle/`, `build/`, `.idea/`, compiled files such as `.class`) and any tool-specific runtime directories named in the assignment sheet. Filename: `iase26_assignmentNN_<names>.zip`, where `<names>` is `<Lastname>_<Firstname>` for an individual submission or `<Lastname1>_<Firstname1>__<Lastname2>_<Firstname2>` for a group submission.
- Submitted code must **build and run as-is** using the toolchain and commands stated in the assignment sheet. Code that does not compile or is not executable will be graded with **0 points**, unless otherwise stated in the exercise.
- If you use **Windows**, we strongly recommend **WSL** for the technical exercises. For shell scripts shipped with assignments, **Git Bash** (bundled with Git for Windows) also works. The scripts shipped with assignments are bash scripts and require `bash` and `zip`, which neither `cmd.exe` nor PowerShell provides by default. We cannot test our code on Windows; if you run into issues, please let us know.
- Familiarize yourself early with the required **dependencies and tools** and their installation. We provide `mise.toml` files with the assignments to simplify dependency installation (see the documentation of **mise**). You can also install the dependencies manually, but we do not recommend it. For some exercises, you will also need a local **Docker** installation. Alternatively, you can install Rancher Desktop. As an IDE, we recommend **IntelliJ IDEA**. You can also use other editors such as Visual Studio Code with appropriate extensions, but we cannot provide support for those. All examples and instructions are based on IntelliJ. For mise, there are various **IDE plugins**.